

# Automated observability with Puppet in a zero-trust environment

VoxConf 2026

# Haroon Rafique

Manager

Development and Operations

Student Information Systems, ITS

***“If you want everything to be familiar, you will never learn anything new, because it can’t be significantly different from what you already know.”***

# The Toil Trap

- Manual monitoring target updates
- Config drift from reality
- Onboarding friction due to cert. management
- YAML sprawl

# Zero-Toil Architecture

- Self-registration
- Automated discovery
- Identity-driven
- Zero-trust

# Implementation Blueprint

- Classify nodes automatically during CSR signing
- Separate logic from data using Hierarchical roles
- Automate service discovery with exported resources
- Secure metrics scraping with Caddy and mTLS
- Standardize monitoring with layered exporter profiles

# Components in This Reference Stack

- OpenVox/OpenVox Server for policy and certificates
- OpenVoxDB for exported resource inventory
- VictoriaMetrics for metrics storage
- Vmagent for metrics scraping
- Grafana for visualization

# Pattern 1: CSR Classification

- Immutable identity requested during provisioning via CSR
- Cryptographic proof stored in the certificate extensions
- Tamper-proof classification secure against agent-side overrides
- Automated autosigning (optionally) enabled by secure policy scripts

What is `pp_role`, and what values should it contain?

- roles
  - `monitoring_scraper`
  - `webserver`

Corresponding Hiera data:

```
roles
├─ monitoring_scraper.yaml
└─ webserver.yaml
```



# Pattern 2: Pure Data Roles

- CSR Extension attributes

```
/etc/puppetlabs/puppet/csr_attributes.yaml
```

```
1 ---
2 extension_requests:
3   pp_role: webserver
```

- **Zero logic** inside site manifest or node definitions

```
manifests/site.pp
```

```
1 node default {
2   # Lookup classes from Hiera
3   lookup('classes', Array[String], 'unique').include
4 }
```

- **Hiera lookup** determines config based on `pp_role`

```
data/roles/webserver.yaml
```

```
1 ---
2 # Webserver role - Apache with monitoring
3 # Applied to nodes with pp_role=webserver in csr_attributes.yaml
4 classes:
5 - profile::apache
6 - profile::monitoring::apache_exporter
```

- Hiera configuration

```
excerpt from hiera.yaml
```

```
1 hierarchy:
2 - name: "Per-role data (from CSR attributes)"
3   path: "roles/%{trusted.extensions.pp_role}.yaml"
```

# Pattern 3: Auto-Discovery

- **Exported resources** publish state to OpenVoxDB

```
1 @@file { "/etc/vmagent/targets.d/${facts['networking']['fqdn']}_${name}.yaml"
2   ensure => file,
3   content => @("EOT"),
4   # Managed by Puppet
5   - targets:
6     - ${facts['networking']['fqdn']}:9090/${name}/metrics
7     labels:
8       job: ${name}
9       instance: ${facts['networking']['fqdn']}
10      exporter: ${name}
11  | EOT
12  tag      => 'vmagent_target',
13 }
```

- **Dynamic collection** gathers all active targets

```
1 # Collect all exported vmagent targets from other nodes  
2 File <<| tag == 'vmagent_target' |>>
```

# Pattern 4: Zero-Trust mTLS

- **Caddy** acts as a *sidecar* reverse proxy and mTLS server

excerpt from Caddyfile

```
1 # Authenticate via Puppet CA with mTLS
2 tls /etc/caddy/node-cert.pem /etc/caddy/node-key.pem {
3     client_auth {
4         mode require_and_verify
5         trust_pool file /etc/caddy/puppet-ca.pem
6     }
7 }
```

- **Import** metrics exporter configurations

```
1 import /etc/caddy/conf.d/*.caddy
```

- **Proxy** the exporter traffic

excerpt from `apache.caddy`

```
1 handle_path /apache* {  
2     reverse_proxy localhost:9117  
3 }
```

- Only allow **legitimate** scrapers

```
1 @authorized_scrapers_snippet {  
2     expression \  
3         {tls_client_subject} == "CN=vmagent.local"  
4 }
```

- **Block** unauthorized scrapers

```
1 handle {  
2     abort  
3 }
```

# Pattern 5: Layered Observability

- **Common** profile applies to all nodes

```
data/common.yaml  
1 classes:  
2 - profile::base
```

- **Baseline metrics** applied to every node

```
site/profile/manifests/base.pp  
1 class profile::base {  
2   include profile::monitoring::node_exporter  
3 }
```

- **Role-specific** exporters are added based on workload

```
roles/webserver.yaml  
1 classes:  
2 - profile::apache  
3 - profile::monitoring::apache_exporter
```

# The Automated Pipeline

## Host Lifecycle

- **Host boots** and starts OpenVox agent
- **Agent submits** CSR with role intent
- **Host applies** role-specific profiles
- **Exporters publish** scrape endpoints

## System Response

- **CA auto-signs** based on secure policy
- **Hiera maps** data classes to the host
- **OpenVoxDB stores** the exported assets
- **Vmagent polls** the target automatically



# Security Guardrails For Production

- **Strict validation** of role-bearing CSRs.
- **Role restrictions** to prevent privilege escalation.
- **Least-privilege paths** for authorized scrapers only.
- **Continuous auditing** of exported resources.

# Does It Work?

- **Initial State:** Verify current targets are up.
- **The Trigger:** Provision a new webserver node.
- **The Magic:** Watch Puppet execute automatic classification.
- **The Result:** The new scrape target populates automatically.
- **The Payoff:** View the live Grafana dashboards.

# Demo

VoxConf 2026

# Operational Impact

- **Scales infinitely** from isolated labs to massive production fleets.
- **Adapts dynamically** to volatile node churn and auto-scaling.
- **Accelerates visibility** by establishing instant scrape targets.
- **Minimizes effort** by keeping operational overhead flat as the nodes scale.

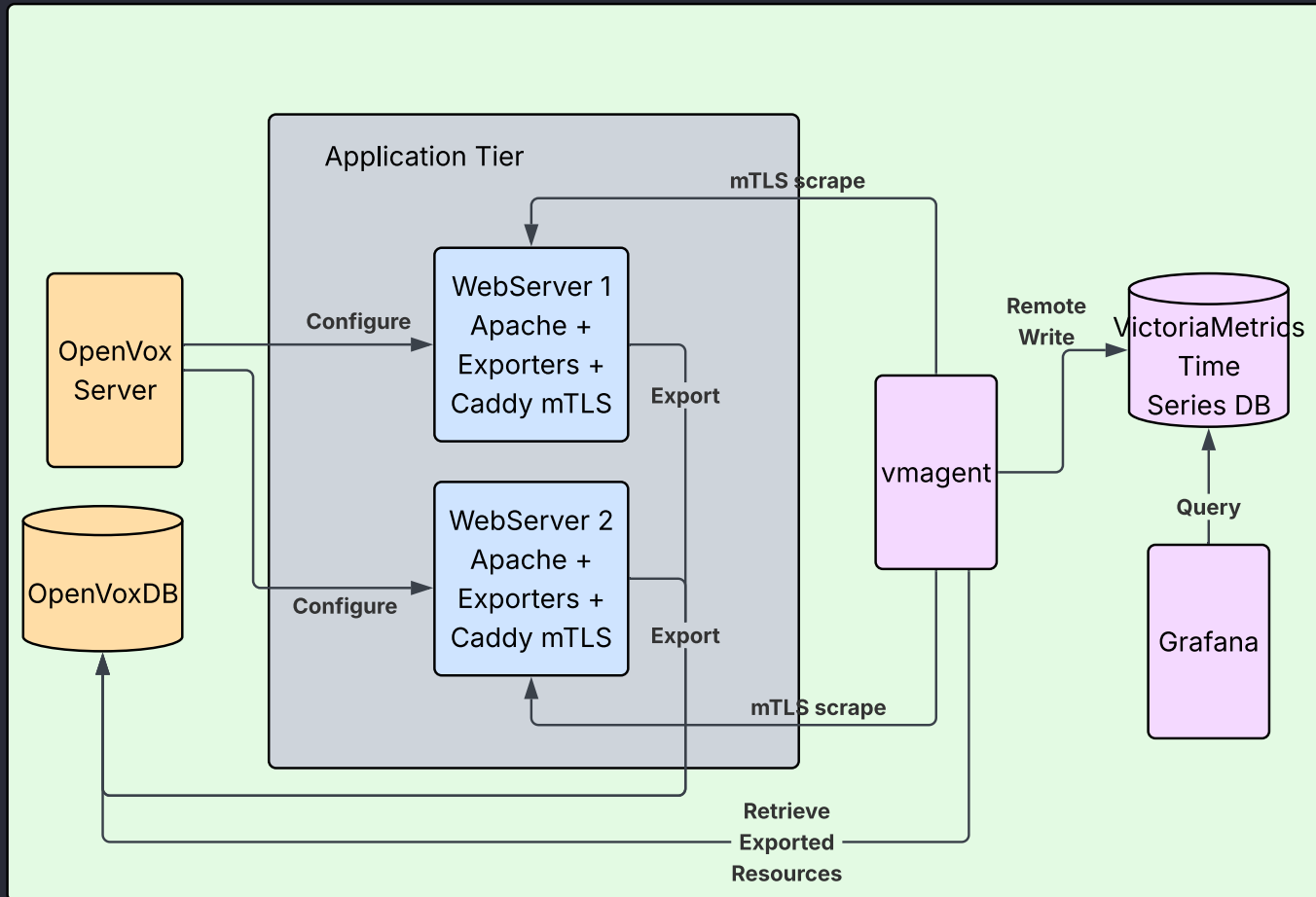
# Engineering Anti-Patterns

- **Mutable scripts** installing runtime packages
- **Floating tags** using `latest` in production
- **Hard-coded credentials** in compose files
- **Copy-pasted logic** repeated across services
- **Ungoverned attributes** trusted without policy controls

# Key Takeaways

- **Automate observability** from the exact second a node boots
- **Eliminate bottlenecks** by moving classification to CSRs
- **Drop manual** updates using native exported resources
- **Recycle existing security** to achieve zero-trust mTLS
- **Scale your infrastructure** without increasing team toil

# Bonus Slide: System Architecture



# Questions?



# Git Repo



VoxConf 2026